

CPE 334 Project Deliverable 1

Engineering baseline and working Sprint 1

Due 8 November 2026 at 23:59 | 80 points (8% of Grade) | Semester 1/2026

1. Purpose and assessment

This deliverable proves that your team can **plan the product and engineer/build Sprint 1**. Establish the engineering baseline for the whole product, then implement a small, useful, tested vertical slice. This release becomes the starting point for Labs 5–7.

Work with the same two students and repository used for the group project. Use **group_work_specs.docx** as the master specification, including its theme assignment and Sprint 1 examples.

Part	Points	What you must demonstrate
A. SRS and product scope	10	Actors, scope, FR/BR/NFR and decisions
B. Feature inventory and traceability	10	Requirements mapped to features and sprints
C. System-Level SDS	15	Architecture, data, UI/API, access and deployment plan
D. Three-sprint plan	5	Goals, allocation, dependencies and feasible scope
E. Sprint 1 engineering contracts	20	Feature design, AC, STS, DoD and version history
F. Working Sprint 1 and verification	20	Domain workflow, tests, access and reviewed integration
Total	80	

2. Required Sprint 1 outcome

By the deadline, release a working increment on **main** with documented local setup. It must include:

- A complete domain journey through the UI, API/application layer and database.
- Real login, logout and current-user identity, and at least two meaningful user roles.
- A meaningful domain transaction or state transition and business-rule validation, comparable to your theme's Sprint 1 example.
- Automated tests for the domain rules, persistence and access restrictions, plus selected UI/end-to-end journeys.
- GitHub Issues, feature branches, teammate-reviewed PRs, sprint staging/integration and an accepted release on main.

Docker, completed CI/CD and GCP deployment are due in Lab 5. At this checkpoint, retain local build/test results linked to the release commit and design the application for later deployment. Seeded test accounts are sufficient. Self-registration, password recovery and advanced account administration are not required.

3. Required engineering work

Parts A–D describe the whole product. Parts E–F specify and verify the Sprint 1 increment..

3.1. Part A SRS and product scope

10 points | Suggested artifact: docs/srs.md

- Describe the stakeholder problem, chosen context, actors and their responsibilities. State the product scope and explicit exclusions.
- Identify Functional Requirements (FR) and Business Rules (BR) with stable IDs. Define observable behavior and meaningful domain rules, rather than generic claims such as “manage records.”
- Define important Non-Functional Requirements (NFR) with measurable targets and a practical verification method. Allocate their verification to an appropriate sprint.
- Record significant assumptions, design decisions, questions and their resolution.

Example of a testable rule: “A customer cannot create a second active booking for an occupied exclusive slot.” Example of measurable verification: “An unauthenticated request to a protected endpoint is rejected without returning private data,” verified by an API test.

3.2. Part B Feature inventory and traceability

10 points | Suggested artifacts: docs/features.md and docs/traceability.md

List the features with stable IDs, short descriptions, related requirements, dependencies and planned sprints. Cover every baseline FR, BR and important NFR. A cross-cutting requirement may link to several features or a named system responsibility; identify which sprint establishes and verifies it.

Requirement	Feature or responsibility	Sprint and verification
FR-01	F-01 Create booking	Sprint 1; AC-01 / TEST-01
BR-01	F-01 Prevent duplicate booking	Sprint 1; AC-02 / TEST-02
NFR-01	F-02 Protect private booking data	Sprint 1; AC-03 / TEST-03

This is an illustrative appointment-system table. Use your own IDs and rules. **Requirements in later sprints need a feature/responsibility and sprint allocation now; they do not yet need fully elaborated feature AC and executable tests.** Show uncovered requirements and approved changes explicitly.

3.3. Part C System-Level SDS

15 points | Suggested artifact: docs/system-sds.md

Architecture and data

- Explain the selected stack and component responsibilities across UI, API/application layer and database. Identify how they communicate and where business rules are enforced.
- Define the major data entities, fields, relationships, important constraints and ownership links. Describe database setup, migrations and reproducible seed data.
- Use a diagram when it clarifies the architecture or data model. There is no required quota of UML diagrams.

UI API and access conventions

- Define the main screens/navigation and shared UI behavior, including validation, loading, empty and error states.
- Define the API approach: endpoint conventions, request/response structure, validation and safe error handling. Feature contracts provide the detailed endpoints.
- Provide a role/permission matrix showing who may perform each protected operation and which records they may access. A hidden frontend control is not backend authorization.
- Describe credential handling, password hashing if passwords are managed locally, session/token creation and expiry, current-user retrieval and logout behavior.

Configuration and delivery design

Explain environment variables, secret handling, database configuration and the planned GCP deployment components. The product must be suitable for Docker packaging and durable storage; **actual cloud setup comes in Lab 5**. Keep real secrets out of Git and client code.

3.4. Part D Three-sprint plan

5 points | Suggested artifact: docs/sprints/plan.md

For each sprint, state its goal, included feature IDs, dependencies and completion outcome. Collectively, the three sprints must cover the product baseline.

Sprint	Required outcome	Checkpoint
1	Small working domain slice with authentication, permissions and persistence	Deliverable 1; 8 Nov
2	Completed core expansion under engineering contracts	Deliverable 2; 29 Nov
3	Remaining features, integration and product hardening	Progress at 29 Nov; complete by 13 Dec

3.5. Part E Sprint 1 engineering contracts

20 points | Suggested artifacts: docs/sprints/sprint-1/ with feature specifications and STS

Prepare and version the engineering contract for **each Sprint 1 feature before implementing it**. The contract is the feature design, acceptance criteria, test specification and completion evidence. Specifications may evolve, but revisions must remain visible and justified.

Feature design and acceptance criteria

- Identify the feature and linked FR/BR/NFR. Describe behavior, workflow, dependencies, business rules, validation, failure/boundary cases and permissions.
- Specify applicable UI behavior, API methods/paths and request/response/error shapes, and data fields/constraints/migrations. State when a layer needs no change.
- Give acceptance criteria stable IDs and observable expected outcomes. Include relevant negative and boundary behavior, and role restrictions.

Software Test Specification and Definition of Done

For each planned test, record its ID, linked AC, test level, preconditions/input, action and expected result. Map all Sprint 1 AC to suitable verification. Unit tests should cover important logic; API/database integration tests should cover persistence and permissions; selected UI/end-to-end tests should cover critical user journeys.

Test and AC	Input or precondition	Expected result
TEST-02 / AC-02	Exclusive slot already has an active booking; submit another booking	Reject the request; no second active booking is stored.
TEST-03 / AC-03	User A requests User B's private booking through the API	Reject access without returning private booking data.

Define the Product Definition of Done: required behavior implemented; planned verification executed; meaningful tests passing; review and integration complete; setup/documents current; and incomplete work disclosed. Completion must be supported by evidence, not an AI agent's claim.

Keep specification, test-design and implementation history. **Use TDD for meaningful rule work: retain evidence that the intended behavior test failed before correction and passed afterward.** A specification-only commit followed by implementation is a useful way to make the sequence visible; no special commit-count requirement applies.

3.6. Part F Working Sprint 1 and verification

20 points | Artifacts: released application, tests, README, GitHub Issues and reviewed PRs

Product behavior and tests

Demonstrate the complete Sprint 1 domain journey, then retrieve its persisted result. Demonstrate a meaningful rule rejection and a forbidden operation. Test actual API/database integration.

Include automated checks for valid/invalid login, unauthenticated protected requests, and forbidden role operations. Demonstrate logout according to the chosen session/token design. Use real backend enforcement, not a client-selected user ID or role selector.

Run the required build and test commands on the submitted release. Retain test names/files, counts, results, command and tested commit. Inspect assertions and failures yourself. Report failed/skipped tests; do not disable or weaken required tests to make the result appear complete. Manual checks may supplement meaningful automated verification.

Reviewed integration and team participation

- Link each completed Issue to its feature/contract, branch and PR. Implement features on branches from sprint staging.
- Have the teammate review the PR into sprint staging. Record a specific check, question, finding or reason for approval; authors do not approve their own work.
- Integrate the increment, run integration/system tests and acceptance checks, then merge the reviewed staging release PR into main.
- Record the release tag/commit. Both students must have meaningful participation; identify pair work in Issues/PRs and a short contribution table.

An equal number of commits is not required. Both students must understand the design, tests and released behavior.

4. What to submit

Submit **one self-contained PDF per team through LEB2 by 8 November 2026 at 23:59**. Put the complete engineering documents and selected execution evidence directly in the PDF. A TA should be able to grade Parts A–F without opening separate specification files.

Keep the engineering documents, source code and tests in the GitHub repository as well. At the beginning of the PDF, provide both students' names and IDs, the selected theme/context, **one repository URL and the submitted release tag/commit**. The repository is available for targeted verification; it is not a substitute for the required PDF contents.

4.1. Organize the PDF into Parts A to F

Use the headings below in the same order as shown here. Include a table of contents with page numbers and keep text, diagrams, tables and screenshots readable at normal viewing size. There is no fixed page limit; include complete engineering content without duplicating it unnecessarily.

Part	Points	Include directly in the PDF
A	10	Complete SRS: stakeholder/context, actors, scope/exclusions, numbered FR/BR/NFR, assumptions and decisions.
B	10	Complete feature inventory and requirements-to-feature-to-sprint traceability table. Show coverage gaps and approved changes.
C	15	Complete System-Level SDS: architecture/stack, major data model/diagrams, UI/API conventions, roles/ownership, authentication, configuration and deployment plan.
D	5	Three-sprint plan: goals, feature IDs, dependencies and completion outcomes. Explain the scope of Sprint 1.
E	20	All Sprint 1 feature engineering contracts: feature SDS, applicable UI/API/data specifications, AC, STS and Product Definition of Done.
F	20	Selected program demonstrations, build/test results, test coverage/results table, Git workflow/history evidence, contribution table, and known limitations.
Total	80	

4.2. Show selected execution evidence in Part F

Program demonstrations

Include selected screenshots or screen dumps showing the following scenarios:

- A complete normal domain journey through the UI, API and database, followed by retrieval of the saved result.
- A meaningful business-rule or boundary rejection, with the expected failure shown.
- Login/logout and a forbidden role or ownership operation. Include backend/API test evidence for protected operations; hiding a button does not prove authorization.

Label each scenario with its feature/AC IDs, user role, starting conditions, brief steps, expected result and actual result. Identify the release commit once for the demonstration set, and state any exception.

Screenshots should show the relevant content clearly. A short sequence may demonstrate several related criteria.

Build and automated test results

Include the build/test commands, tested commit and readable execution output showing the passing, failing and skipped counts. Provide a results table mapping each Sprint 1 AC to its planned test ID, actual test file/name and result. Clearly distinguish automated tests from manual checks.

Embed **two short excerpts of actual automated tests**: one for an important domain rule and one for a role/ownership restriction. Show the relevant setup and assertion, not entire source files. Include their execution results. For meaningful TDD work, show the intended behavior test failing before implementation/correction and passing afterward, with the relevant versions identified.

Engineering workflow and history

Include selected evidence for **two completed Issues and their PRs**, showing the feature/contract IDs, branch names, author/reviewer, a substantive review, and integration into sprint staging and main. A compact table plus readable GitHub/history screenshots is sufficient. Include the final board status and release reference.

Show a concise history sequence identifying when the sampled contracts/test design were recorded and when implementation followed. Preserve commit IDs and dates. Include both students' contributions, a brief AI-use declaration, and known incomplete work or failed/skipped tests. Full prompt transcripts and long reflections are not required.

4.3. Include a short navigation table

At the beginning of the PDF, provide one navigation row for each part A–F with its page range and relevant IDs. Use page references within the PDF rather than external artifact links. For example: “Part E, pages 12–18: F-01, AC-02, TEST-02; execution evidence in Part F, page 22.” Use your actual page numbers.

TAs will read and sample evidence from the PDF. They may open the repository to investigate unclear or inconsistent evidence or verify a selected claim. Keep the repository accessible and tied to the submitted release. Later commits must not silently replace the assessed version.

4.4. Retain reproduction instructions and safe test access

Keep the README current with dependencies, environment-variable examples, database setup/migrations/seed, and application/build/test commands. Include a brief setup summary in Part F so the assessment is understandable without opening the README. Use synthetic demo data. Provide safe local test accounts through the private course handoff or documented local seed procedure; do not put real passwords, tokens or production secrets in the PDF or Git.

A separate demo video, GCP URL, completed Docker setup and CI/CD pipeline are not required for Deliverable 1. Deployment is assessed in Lab 5; the final project requires its own demonstration video. The selected screenshots and test evidence above document the local Sprint 1 release.

5. Submission checklist

- The PDF contains complete Parts A–E and selected execution/process evidence in Part F.
- The contents/navigation table identifies A–F and their page ranges.
- All Sprint 1 contracts, AC, STS and DoD appear directly in the PDF.

- Program screenshots demonstrate the normal workflow, persisted retrieval, rejection and access restrictions with captions.
- Build/test output, the AC-to-test results table and selected test excerpts identify the tested versions.
- Selected Issue/PR/review/history evidence and both students' contributions are readable in the PDF.
- The PDF identifies the repository and immutable release; the repository remains accessible and reproducible.
- Incomplete work and failed/skipped tests are disclosed; no real secrets are included.